

.

Generare Thumbnail-uri pe baza emotiilor

Autor: Hatu Denis-Alexandru, 332AB

An: 2024-2025

Cuprins

1. Introducere.....	3
2. Suport tehnic.....	3
3. Realizarea aplicatiei in Python.....	4
4. Mod de utilizare.....	9
5. Concluzii.....	9
6. Referinte	10

1. Introducere

În era în care totul este despre realizare de conținut, imaginile miniaturale, cunoscute sub forma de thumbnail-uri joacă un rol esențial în atragerea utilizatorilor să vizioneze respectivul clip. Pentru a realiza acest lucru, am compus diferiți pași de resampling a pozelor (în sensul de eliminare a pozelor care nu corespund anumitor cerințe) împreună cu un clasificator de imagini:

- Resampling al cadrelor: extragerea cadrelor din multitudinea de imagini, folosind criterii matematice precum claritatea imaginii, detalii de lumină, prezența fetelor.

- Detectarea fetelor: folosind în prima instanță MediaPipe pentru viteză și YOLO pentru a elimina falsuri pozitive.

- Clasificarea fetelor: antrenarea unui CNN cu setul de date fer2013.csv pentru a-mi prezice pozele mele. Implementarea unui CNN de la 0 a fost făcută în contextul computațional al calculatorului de pe care am dezvoltat aplicația, folosind un ResNet50, probabil nu terminăm.

Tema aleasă îmbină cunoștințe din mai multe arii: procesarea imaginilor, analiză video, noțiuni din matematică, învățare automată și detectia de obiecte. Aplicația este utilă în mod special pentru creatorii de conținut aflați la început de drum, care nu dispun de abilități de edit sau de buget pentru a colabora cu editori.

Obiectele proiectului:

- Realizarea unei aplicații care clasifică corect emoțiile dintr-o poză și să genereze automat un folder specific pentru fiecare emoție în care se pot găsi posibilele thumbnail-uri.

- Integrarea modelelor MediaPipe și YOLO pentru procesare rapidă a reperelor feței, recunoașterea fetelor

- Selecție pozelor pe baza unor metrici (claritate, prezența feței)

- Oferirea unui produs software open-source care poate fi extins în viitor.

2. Suport tehnic

Aplicația este realizată exclusiv în Python, folosind ca și mediu de dezvoltare Visual Studio Code, integrat într-un mediu virtual. Datele de intrare sunt diverse: fișiere .csv care servesc drept bază de date, imagini pentru antrenarea rețelei YOLO, fișiere .h5 pentru predicția asupra setului meu de poze și modele YOLO pre-antrenate salvate în format .pt.

Printre bibliotecile cele mai frecvent utilizate se numără OpenCV (pentru procesare de imagine și video), Decord (pentru extragerea eficientă a cadrelor din video) și MediaPipe (pentru detectarea și analizarea trăsăturilor faciale).

Aspectele multimedia predominante in acest proiect:

Folosirea diverselor de tipuri de imagine: RGB, respectiv Grayscale pentru modelele antrenate pentru a reduce dimensiunea si costurile de antrenare.

Reprezentarea video: cadre, fps (cadre pe secunda) destul de relevante, avand in vedere ca reprezinta componenta de baza pentru clasificare.

3. Realizarea aplicatiei in Python

Prerequisites:

`!pip install tensorflow opencv-python mediapipe torch ultralytics decord cuda numpy pillow scikit-learn`

In primul rand, avand in vedere ca trebuie sa clasificam thumbnail-uri, trebuie sa impartim videoclipul pe care il alegem in mai multe cadre. Pentru a face acest lucru puteam folosi OpenCV, doar ca decord este mult mai eficient conform benchmark-urilor.

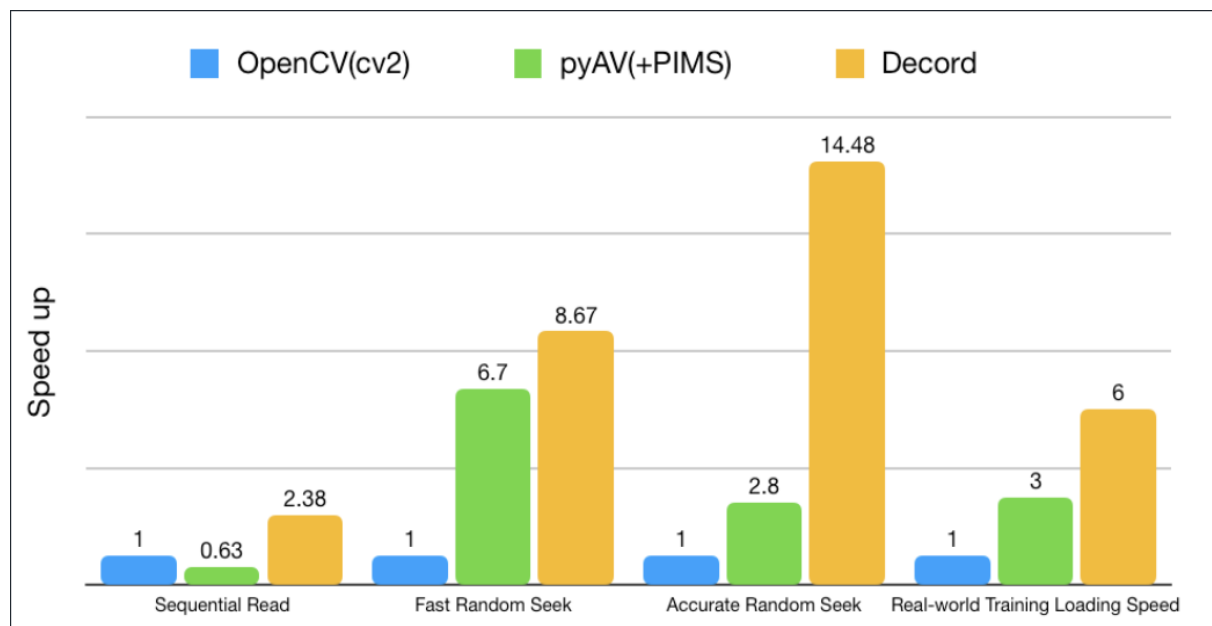


Figura 1

Citesc video-ul prima oara, extrage batch-urile conform listei 'frame_indices' folosita pentru a indica la ce interval si ce cadre vor fi selectate din videoclip. Aceste poze sunt stocate intr-un prim folder 'test_frames'.

Pentru ca extragerea tuturor frame-urilor si iterarea prin ele este un proces costisitor am decis sa folosesc ThreadPoolExecutor din biblioteca concurrent.futures.

Pentru a elimina rapid pozele care nu contin fete am ales sa folosesc MediaPipe care este un API foarte eficient care face clasificarea pe baza a 6 puncte de interes ale fetei si suporta detectie multipla a fetelor. Pentru a fi sigur ca elimin pozele care nu contin fete am considerat o limita de 0.7.

Astfel, raman cu pozele care nu contin fete, dar, din cauza faptului ca nu este foarte robust (precizie scazuta din cauza vitezei, sensibila la unghiul fetei), am decis sa folosesc un model YOLO pentru a ma asigura ca nu exista falsuri pozitive precum cutii, tablouri, jucarii.

Folosesc o functie `prepare_dataset()` care pregateste un set de date pentru antrenarea unui model YOLOv8. Ca si set de date am folosit Widerface (vezi referinte bibliografice) datorita volumului mare de date pe care il are, fiind sigur ca voi avea o acuratete buna pe imaginile mele.

Creez directoarele de train si test, impart datasetul in train si test, copiez imaginile si label-urile in directoarele pe care le-am creat anterior si creez un fisier `.yaml` specific unui model YOLO in care retin calea catre imaginile de antrenare, testare, numarul de clase si numele clasei.

Cu setul de date corect impartit, urmeaza antrenarea modelului YOLO si folosirea acestuia pe pozele mele cu o limita inferioara mare, chiar daca s-ar putea pierde poze mai departate, dar, se vor elimina cu siguranta falsurile pozitive.

Dupa eliminarea pozelor care sunt fals pozitiv si pozele care nu contin fete, ar fi bine ca pentru logica programului nostru sa luam in considerare poze care nu contin mult sunet, miscare, fie in jurul fetei, fie in general in poza. Pentru acest aspect m-am folosit de notiuni matematice esentiale in analiza de poze, anume Transformata Fourier si Laplacianul.

Pentru procesare mult mai rapida prima oara am ales sa folosesc Laplacianul simplu si cu kernel 1D.

In primul rand, sa vorbim putin despre aspectele matematice care sunt folosite in filtrul Laplacian. Laplacianul este un filtru derivativ de ordinul 2, care măsoară cât de repede se schimbă gradientul unei imagini. Folosit pentru a identifica margini si detalii fine. O imagine clara va produce valori ale laplacianului ridicate, iar o imagine blurata produce un laplacian aproape uniform.

$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Figura 2

Daca ne raportam la derivata in raport cu x:

$$\frac{\partial^2 f}{\partial^2 x} = f(x + 1, y) + f(x - 1, y) - 2f(x, y)$$

Figura 3

Si derivata in raport cu y:

$$\frac{\partial^2 f}{\partial^2 y} = f(x, y + 1) + f(x, y - 1) - 2f(x, y)$$

Figura 4

Iar apoi le adunam si obtinem:

$$\nabla^2 f = [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1)] - 4f(x, y).$$

Acum, daca luam in calcul un filtru de 3*3 pentru o imagine reprezentata in coordonate(x,y), poate fi reprezentata astfel:

x-1,y-1	x,y-1	x+1,y-1
x-1,y	x,y	x,y+1
x-1,y+1	x,y+1	x+1,y+1

Daca vom pune coeficientii aferenti ecuatiei, vom obtine urmatoarea matrice:

0	1	0
1	-4	1
0	1	0

Functia ‘is_motion_detected_using_laplacian()’ converteste imaginea in grayscale pentru a identifica mai bine detaliile, aplic Laplacianul si calculam varianta rezultata. Daca vom avea varianta mica, imaginea va fi blurata.

Alegem varianta deoarece indica dispersia valorilor. Valorile dispersate, multe detalii => clar ; In opozitie, valori apropiate, putine margini => blur. Alegem o limita superioara corespunzatoare si salvam pozele care corespund.

Functia ‘is_motion_detected_using_laplacian_with_kernel()’ foloseste un principiu similar, doar ca am decis sa folosesc un filtru 1D aplicat pe doua directii. Un kernel reprezinta un filtru de marimi reduse care se „plimba” de-a lungul imaginii. Un kernel derivativ este conceput pentru a detecta variatii rapide in intensitate, margini, contururi.

Filtrul pe care il folosesc actioneaza pe 3 pixeli, un pixel central si doi vecini. Daca pixelul e similar cu vecinii, rezultat mic, ceea ce inseamna blur. Folosesc acest filtru atat orizontal, cat si vertical pentru a detecta mai usor comportamentul imaginii.

Aceste variante sunt folosite ca si metode rapid computational, dar care poate esua in anumite metode, datorita faptului ca nu este un proces atat de complex. Aici intervine Transformata Fourier care va fi folosita in caz ca cele doua nu sunt in domeniul pe care eu il doresc.

Prima oara, pentru a avea rezultate bune, aleg doar fetele prin intermediul modelului YOLO deja antrenat. Extrag coordonatele fetei si le convertesc la int pentru a avea o lista de fete detectate. Folosesc functia ‘crop_face ()’ pentru a selecta doar regiunea de interes pe care o

doresc, procesand numai pixelii fetei. Folosesc FFT (Fast Fourier Transform) care in loc sa faca 2 sume cu complexitate $O(M^2N^2)$, o sa aibe $O(MN\log(MN))$.

Vom calcula FFT, apoi vom shifta componenta care contine frecventa zero catre centru pentru a face o analiza mai simpla. Ne folosim de magnitudinea spectrului pentru detectarea de blur. Convertim la scala logartimica, extragem dimensiunile imaginii si centram domeniul de analiza a pozei, eliminand frecventele mici, iar apoi masor practic cat de neteda sau miscata este poza.

Chiar daca am scapat de pozele care nu au fete, neclare, in continuare mai putem avea un scenariu foarte probabil, pozele in care persoanele au ochii inchisi. Pentru acest aspect, ne putem raporta la EAR (Eye Aspect Ratio), pe care il vom calcula pe baza anumitor puncte de interes ale ochilor.

Pentru a extrage punctele de interes ale ochilor avem nevoie de un model, folosirea modelului de 68 de caracteristici ar fi fost prea incert deoarece punctele nu sunt dispersate optim. Avand in vedere acest aspect, am decis sa folosesc face_mesh din cadrul MediaPipe, avand pentru fiecare ochi 6 puncte de interes.

Ca sa observam comportamentul ochiului (deschis/inchis) trebuie sa calculam distanta euclidiană pe cele doua directii, orizontala si verticala. Calculam normele corespunzatoare celor doua directii, apoi pe baza unei limite putem determina daca persoana despre care este vorba are ochii inchisi/aproape inchisi.

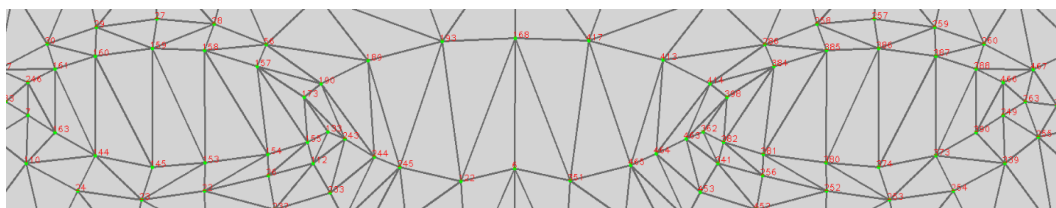


Figura 5

Deoarece vrem sa avem un thumbnail cat mai concret, avem nevoie de a elimina pozele in care nu apare cel mai frecvent personaj. Pentru a realiza acest lucru m-am folosit tot de MediaPipe pentru a extrage reperele faciale ale persoanelor.

Retin reperele faciale ale primei persoane, daca acele repere faciale sunt predominante in folder(diferenta dintre reperele etalon si cele de comparat este mica), atunci vom lua in calcul persoana ca fiind predominanta. Apoi vom itera din nou, iar daca lungimile caracteristicilor, sau daca fata nu este detectata corect se va returna infinit.

Daca se detecteaza si totul este ok, se calculeaza distanta euclidiană, daca aceasta este intr-un domeniu, atunci totul este in regula, daca nu, imaginea respectiva se va exclude pentru a usura alegerea thumbnail-ului.

Pentru ca am folosit suficieni pasi pentru resampling, acum putem sa clasificam pozele ramase in expresii faciale, precum : furie, dezgust, frica, fericire, tristete, neutru sau surprins. Pentru a realiza acest task m-am folosit de o retea neuronală in ansamblu cu un set de date potrivit pentru a-mi prezice datele.

Ca si set de date, am ales fer2013.csv (disponibil pe Kaggle :vezi referinte bibliografice) in ciuda caracteristicilor reduse (numar mic de sample-uri in comparatie cu alte seturi de date folosite pentru clasificare) si a imaginilor reduse (foloseste imagini de 48x48 care sunt utile pentru costul computational al retelei, dar va fi foarte probabil ca reseaua sa produca metrice bune >85%), am obtinut o precizie de 67% fata de 75% de pe Kaggle.

Pentru ca lucram cu valori ale pixelilor, numarul de sample-uri reduse si imagini de dimensiuni reduse, good practice este sa aplicam normalizari asupra datelor si sa le scalam inainte de impartirea in seturi de antrenare si de test si de a construi reseaua.

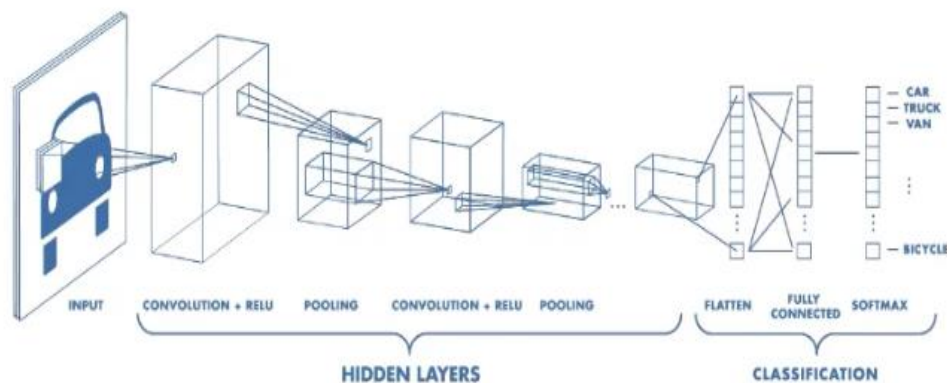


Figura 6

Un CNN reprezinta un tip de retea neuronală specializată în procesarea imaginilor cel mai des. Elemente esențiale:

- Convolutii: detectează caracteristici locale (muchii, texturi)
- Pooling:
- Dense: efectuează clasificarea pe baza caracteristicilor extrase
- Regularizare (Dropout, Batch Normalization): ajută la stabilirea și generalizarea antrenamentului.

Ca și input folosesc imagini de 48x48 pe un singur canal în scalate la $[-1, +1]$ care sunt trecut prin diverse blocuri care au o logică similară. Folosesc un bloc de convoluție cu număr gradual de filtre de 3x3, activez cu ReLU (introduce non-linearitate), stabilizez distribuția activărilor, scade rezoluția la jumate (24x24) și previn overfitting prin Dropout(x). Ultimul strat cu 7 neuroni reprezentând câte o clasă, activare softmax pentru probabilități.

Folosesc adam optimizer, rata de învățare mică, epsilon mic pentru stabilitate numerică. Utilizez metrice de oprire a rețelei neuronale precum `early_stop` și `reduce_learning_rate`.

`EarlyStopping` oprește modelul dacă `val_loss` nu se modifică de-a lungul ultimelor x epoci și reține cele mai bune ponderi. `ReduceLROnPlateau` monitorizează dacă `val_loss` nu scade în x epoci, reduce LR de 10x, până la minim $1e-7$.

Deoarece am poze front-face, unde imaginile sunt expresive si foarte asemanatoare am decis sa folosesc procesul de augmentare a datelor pentru a prevenii overfitting-ul. M-am utilizat pe biblioteca tensorflow pentru a face asta si am facut diverse operatii (rotatie,flip..).Din cauza costurilor computationale am batch size 64 antrenat de-a lungul a 150 de epoci.

4. Mod de utilizare

Clonam repo-ul:

```
git clone https://github.com/deniz2104/Generate-Thumbs-Based-On-Feelings.git
```

Cream un mediu virtual si instalam dependintele:

```
python3 -m venv venv
```

```
source venv/bin/activate    Linux/Mac
```

```
venv\Scripts\activate      Windows
```

pip install -r requirements.txt (unde requirements.txt contine bibliotecile pe care le-am mentionat la inceputul punctului 3)

Se ruleaza scriptul main python .\Thumbnail_generator_based_on_people_feelings.py iar apoi se vizualizeaza folderele create cu numele cate unei emotii distincte.

5. Concluzii

Scopul principal al proiectului a fost dezvoltarea unei aplicații care să permită detectarea fețelor din imagini sau videoclipuri și evaluarea clarității acestora, folosind metode clasice (Laplacian, FFT) și filtre personalizate pentru a genera o imagine miniaturala semnificativa in functie de emotie. Obiectivele propuse au fost atinse în mod complet:

- Detectia fețelor a fost realizată cu succes utilizând modele YOLOv8, care oferă o precizie ridicată.
- Identificarea imaginilor neclare (blurate) a fost implementată folosind atât metode clasice (Laplacian, FFT), cât și o abordare personalizabilă cu kerneluri 1D.
- Preprocesarea și salvarea rezultatelor (crop-uri de față, clasificare blur/clar, salvare CSV și imagini) au fost automatizate, oferind utilizatorului un flux complet.
- Scalabilitatea este susținută prin posibilitatea de a procesa foldere întregi de imagini sau videoclipuri.

Performanță

Metodele folosite oferă un bun compromis între viteză și acuratețe:

- Algoritmul FFT este potrivit pentru imagini cu mult zgomot de fundal, oferind rezultate robuste în medii reale.
- Kernelul personalizat permite adaptarea fină la tipul de aplicație
- Testele au arătat o rată de detecție corectă ridicată, cu timp de procesare scurt.

Impact și relevanță

Proiectul este relevant în contextul actual al dezvoltării aplicațiilor bazate pe computer vision și poate fi extins ușor pentru:

- integrarea într-un API web (pentru aplicații mobile sau de birou),
- antrenarea unui model ML de clasificare pe baza valorilor FFT/Laplacian,
- detectarea emoțiilor sau trăsăturilor faciale (folosind `fer_model.h5` deja antrenat).

Modalități de îmbunătățire ale aplicației

Antrenarea modelului YOLO pentru mai multe epoci pentru o acuratețe mai bună și data augmentation (omite fețele în anumite unghiuri).

Antrenarea unui CNN pentru a prezice dacă ochii sunt deschiși sau închisi. Metoda de abordare curentă merge pe varianta trial and error.

Dezvoltarea unui algoritm mult mai robust atunci când vine vorba de detecția de blur folosind Transformata Fourier.

Din motive pe care nu mi le pot explica, varianta curentă de detecție a persoanelor predominante nu poate analiza pentru mai multe repere faciale.

Antrenarea pe un set de date mult mai mare și mai divers decât `fer2013.csv` alături de o abordare de transfer learning folosind modele mult mai puternice precum VGG11 sau ResNet50.

În concluzie, aplicația dezvoltă cu succes o funcționalitate modernă, adaptabilă și eficientă pentru detecția fețelor și analiza clarității acestora, demonstrând nu doar aplicabilitate practică, ci și potențial de extindere spre aplicații comerciale.

6. Referințe

<https://github.com/dmlc/decord> (ultima oară accesat 23/04/2025)

<https://www.kaggle.com/datasets/lylmsc/wider-face-for-yolo-training?resource=download> (ultima oară accesat 24/04/2025)

<https://www.kaggle.com/code/nhotin/yolotraining> (ultima oară accesat 24/04/2025)

<https://pyimagesearch.com/2020/06/15/opencv-fast-fourier-transform-fft-for-blur-detection-in-images-and-video-streams/> (ultima oara accesat 26/04/2025)

<https://www.cs.bgu.ac.il/~ben-shahar/Teaching/Computational-Vision/StudentProjects/ICBV121/ICBV-2012-1-KerenDamari-BenSimandoyev/Motion%20Blur%20Detection.pdf> (ultima oara accesat 27/04/2025)

<https://medium.com/@sagardhungel/laplacian-and-its-use-in-blur-detection-fbac689f0f88>
(ultima oara accesat 27/04/2025)

<https://www.cl.cam.ac.uk/teaching/1112/CompGraph/notes.pdf> (ultima oara accesat 27/04/2025)

https://raw.githubusercontent.com/google/mediapipe/a908d668c730da128dfa8d9f6bd25d519d006692/mediapipe/modules/face_geometry/data/canonical_face_model_uv_visualization.png (ultima oara accesat 28/04/2025)

<https://medium.com/analytics-vidhya/eye-aspect-ratio-ear-and-drowsiness-detector-using-dlib-a0b2c292d706> (ultima oara accesat 28/04/2025)

<https://medium.com/@ageitgey/machine-learning-is-fun-part-4-modern-face-recognition-with-deep-learning-c3cffc121d78> (ultima oara accesat 29/04/2025)

<https://medium.com/@sifat777/how-to-perform-face-recognition-on-a-large-collection-of-faces-with-python-884b59a5a3dd> (ultima oara accesat 29/04/2025)

<https://medium.com/@prudhvi.gnv/ultimate-guide-for-facial-emotion-recognition-using-a-cnn-f9239fdc63ad> (ultima oara accesat 2/05/2025)

<https://pypi.org/project/face-recognition/>

<https://docs.ultralytics.com/>

<https://pytorch.org/docs/stable/index.html>

<https://ai.google.dev/edge/mediapipe/solutions/guide>

<https://www.tensorflow.org/>